# EDA: Seattle Bus Delay Dataset

Goal: quick health check of the processed dataset and basic signal checks (rain vs. dry, routes, time-of-day).

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

plt.style.use('seaborn-v0_8-whitegrid')
sns.set_context('talk')
pd.set_option('display.max_columns', 30)

DATA_PATH = Path('..') / 'data' / 'processed' / 'dataset.csv'
assert DATA_PATH.exists(), f"Missing file: {DATA_PATH}"

df = pd.read_csv(DATA_PATH)
print(df.shape)
df.head()
```

```
(9389, 23)
```

Out[1]:

| | stop_id | route_id | route_short_name | trip_id | vehicle_id | snapshot_ts | gtfs_ |
|---|---|---|---|---|---|---|---|
| **0** | 1_103 | 1_102745 | G Line | 1_736436399 | 1_6410 | 1772857541 | |
| **1** | 1_85800 | 1_100196 | 331 | 1_778705889 | 1_7203 | 1772857541 | |
| **2** | 1_2850 | 1_100089 | 2 | 1_768042249 | 1_7975173 | 1772896231 | |
| **3** | 1_2850 | 1_100089 | 2 | 1_768040879 | 1_7134 | 1772896231 | |
| **4** | 1_45920 | 1_100031 | 132 | 1_759744689 | 1_8013 | 1772896231 | |

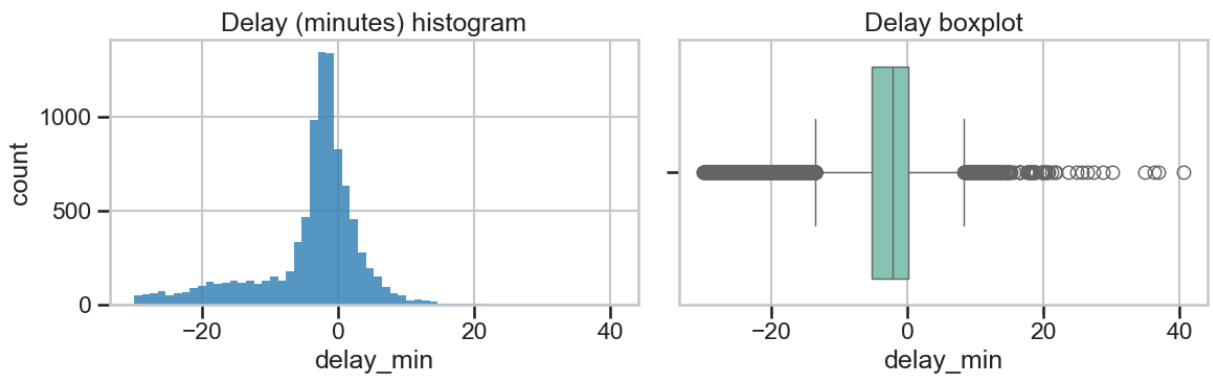
In [2]:
```
# Basic info and missingness
print(df.dtypes)
print('\nMissing values per column:')
print(df.isna().sum())
```

```
stop_id                  object
route_id                 object
route_short_name         object
trip_id                  object
vehicle_id               object
snapshot_ts               int64
gtfs_scheduled_ts       float64
predicted_arrival_ts      int64
delay_min               float64
hour_of_day             float64
day_of_week             float64
is_weekend                int64
is_peak_hour              int64
month                   float64
date                     object
hour_bucket              object
status                   object
temperature_c           float64
precipitation_mm        float64
wind_speed_kmh          float64
visibility_km           float64
weather_code              int64
is_raining                int64
dtype: object

Missing values per column:
stop_id                    0
route_id                   0
route_short_name           0
trip_id                    0
vehicle_id                 0
snapshot_ts                0
gtfs_scheduled_ts          0
predicted_arrival_ts       0
delay_min                  0
hour_of_day                0
day_of_week                0
is_weekend                 0
is_peak_hour               0
month                      0
date                       0
hour_bucket                0
status                     0
temperature_c              0
precipitation_mm           0
wind_speed_kmh             0
visibility_km           9389
weather_code               0
is_raining                 0
dtype: int64
```

In [3]:

```
# Delay distribution
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(df['delay_min'], bins=60, color='#2c7fb8', alpha=0.8)
axes[0].set_title('Delay (minutes) histogram')
axes[0].set_xlabel('delay_min')
```

```
axes[0].set_ylabel('count')

sns.boxplot(x='delay_min', data=df, ax=axes[1], color='#7fcdbb')
axes[1].set_title('Delay boxplot')
plt.tight_layout()
plt.show()

print(df['delay_min'].describe())
```



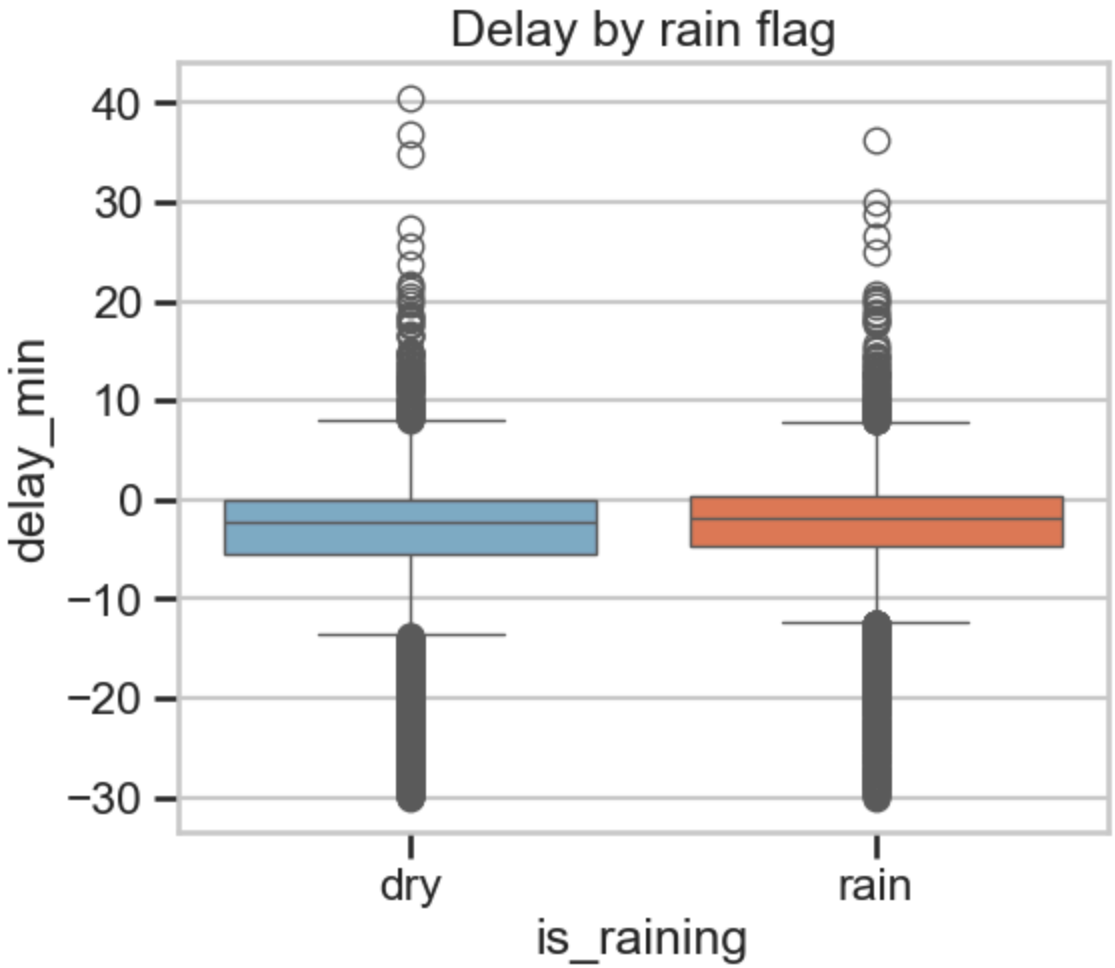

```
count    9389.000000
mean       -3.882150
std         7.785721
min       -29.980000
25%        -5.330000
50%        -2.150000
75%         0.120000
max        40.550000
Name: delay_min, dtype: float64
```

In [4]:

```
# Rain vs. dry comparison
rain_stats = (
    df.groupby('is_raining')['delay_min']
      .agg(['count','mean','median','std','min','max'])
      .rename(index={0:'dry',1:'rain'})
)
print(rain_stats)

plt.figure(figsize=(6,5))
sns.boxplot(data=df, x='is_raining', y='delay_min', palette=['#74add1','#f46
plt.xticks([0,1], ['dry','rain'])
plt.title('Delay by rain flag')
plt.show()
```

```
            count      mean  median       std    min    max
is_raining
dry          5942 -4.050730   -2.32  7.782157 -29.97  40.55
rain         3447 -3.591549   -1.95  7.784421 -29.98  36.25
```

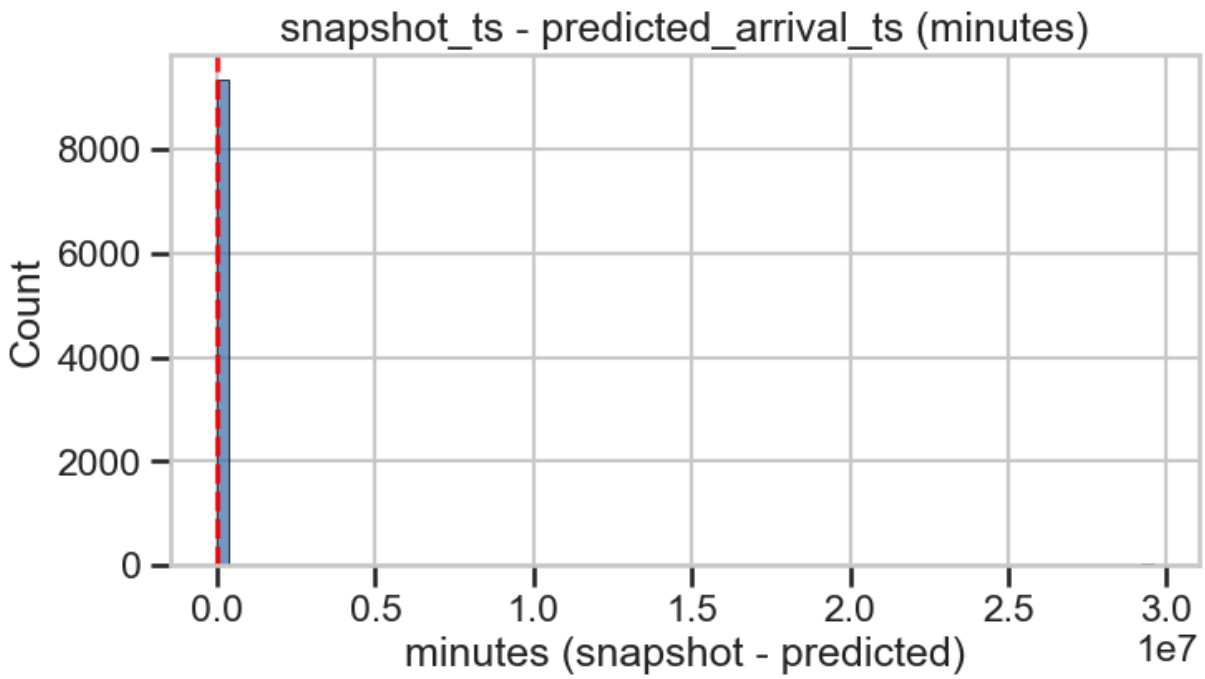
```
/var/folders/5h/3qm4ghsx5335mg6rwp7lw68w0000gn/T/ipykernel_36415/2533216331.
py:10: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the
same effect.

  sns.boxplot(data=df, x='is_raining', y='delay_min', palette=['#74add1','#f
46d43'])
```




In [5]:

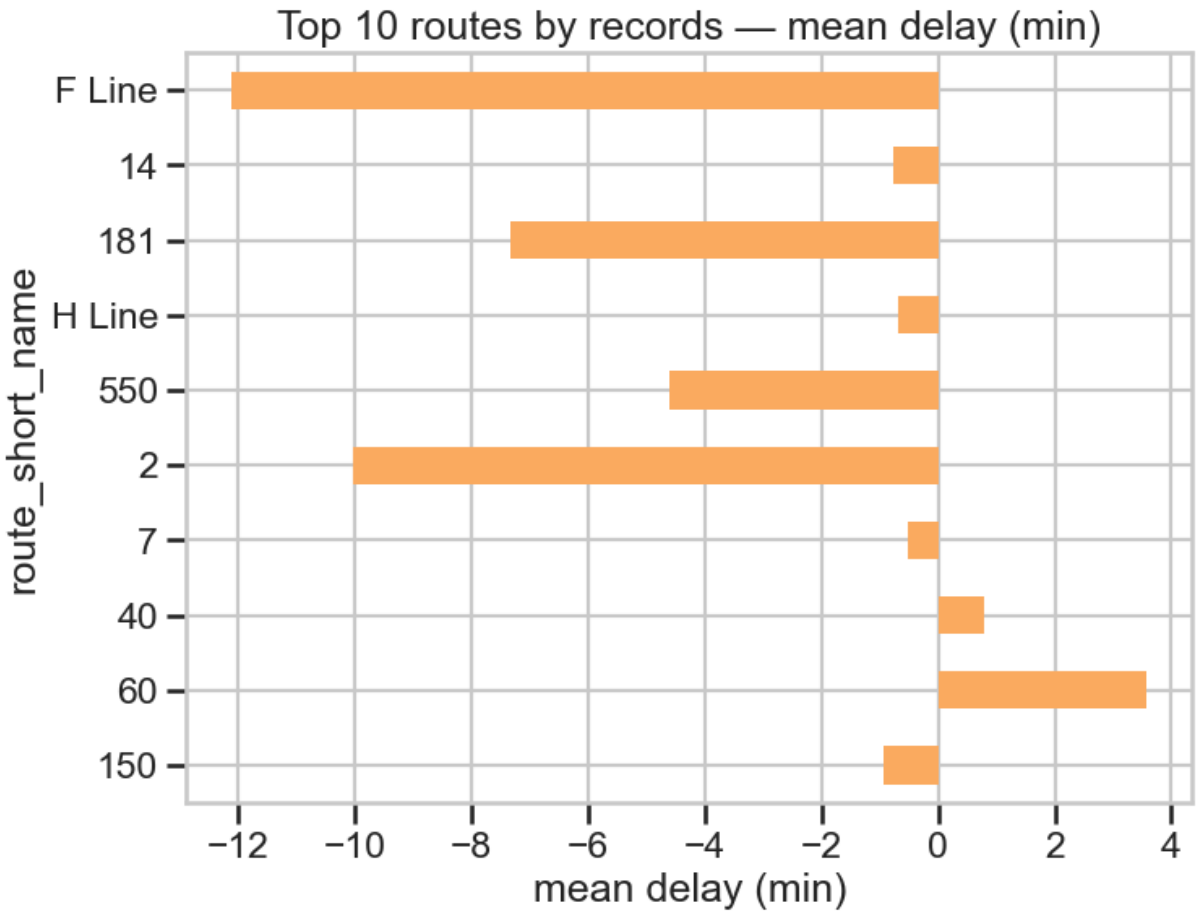
```
# Snapshot vs predicted arrival sanity check
# Expect snapshot_ts ≈ predicted_arrival_ts when bus is at stop.
df['snapshot_pred_diff_min'] = (df['snapshot_ts'] - df['predicted_arrival_ts
print(df['snapshot_pred_diff_min'].describe())

within5 = (df['snapshot_pred_diff_min'].abs() <= 5).mean()*100
print(f"Share within ±5 min: {within5:.1f}%")

plt.figure(figsize=(8,4))
sns.histplot(df['snapshot_pred_diff_min'], bins=80, color='#4575b4')
plt.axvline(5, color='red', linestyle='--', alpha=0.6)
plt.axvline(-5, color='red', linestyle='--', alpha=0.6)
plt.title('snapshot_ts - predicted_arrival_ts (minutes)')
plt.xlabel('minutes (snapshot - predicted)')
plt.show()
```

```
count    9.389000e+03
mean     1.070144e+05
std      1.775269e+06
min     -3.378333e+01
25%     -3.266667e+00
50%     -1.916667e+00
75%     -8.333333e-01
max      2.955544e+07
Name: snapshot_pred_diff_min, dtype: float64
Share within ±5 min: 79.1%
```

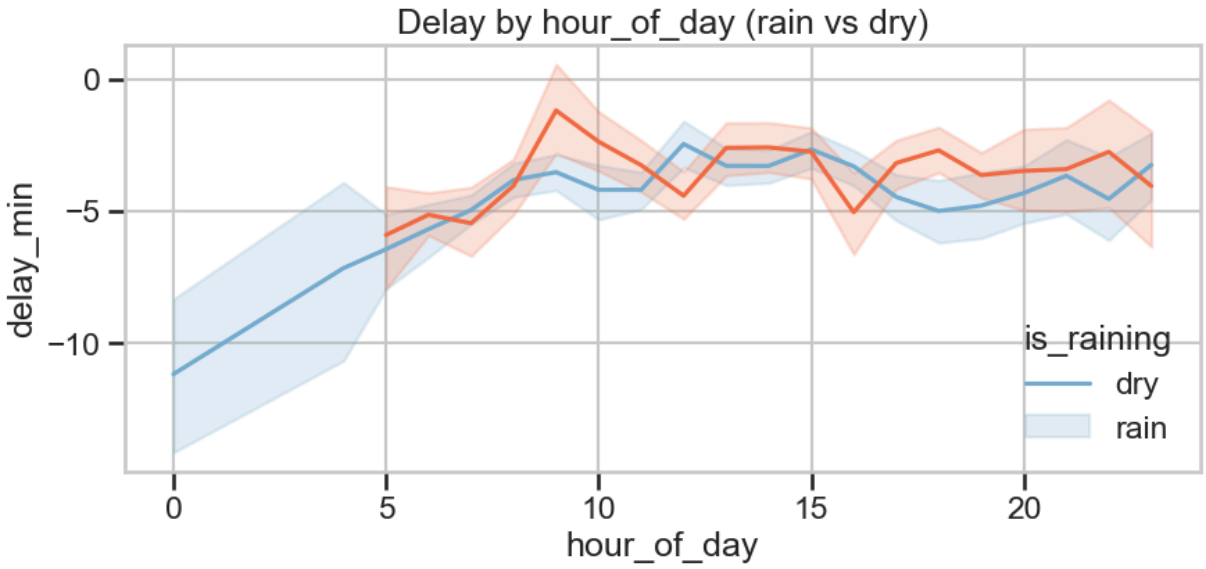



In [6]:
```python
# Route-level volume and mean delay
route_stats = (
    df.groupby('route_short_name')['delay_min']
      .agg(['count','mean','median'])
      .sort_values('count', ascending=False)
)
print(route_stats.head(15))

plt.figure(figsize=(8,6))
route_stats.head(10)['mean'].plot(kind='barh', color='#fdae61')
plt.gca().invert_yaxis()
plt.title('Top 10 routes by records — mean delay (min)')
plt.xlabel('mean delay (min)')
plt.show()
```

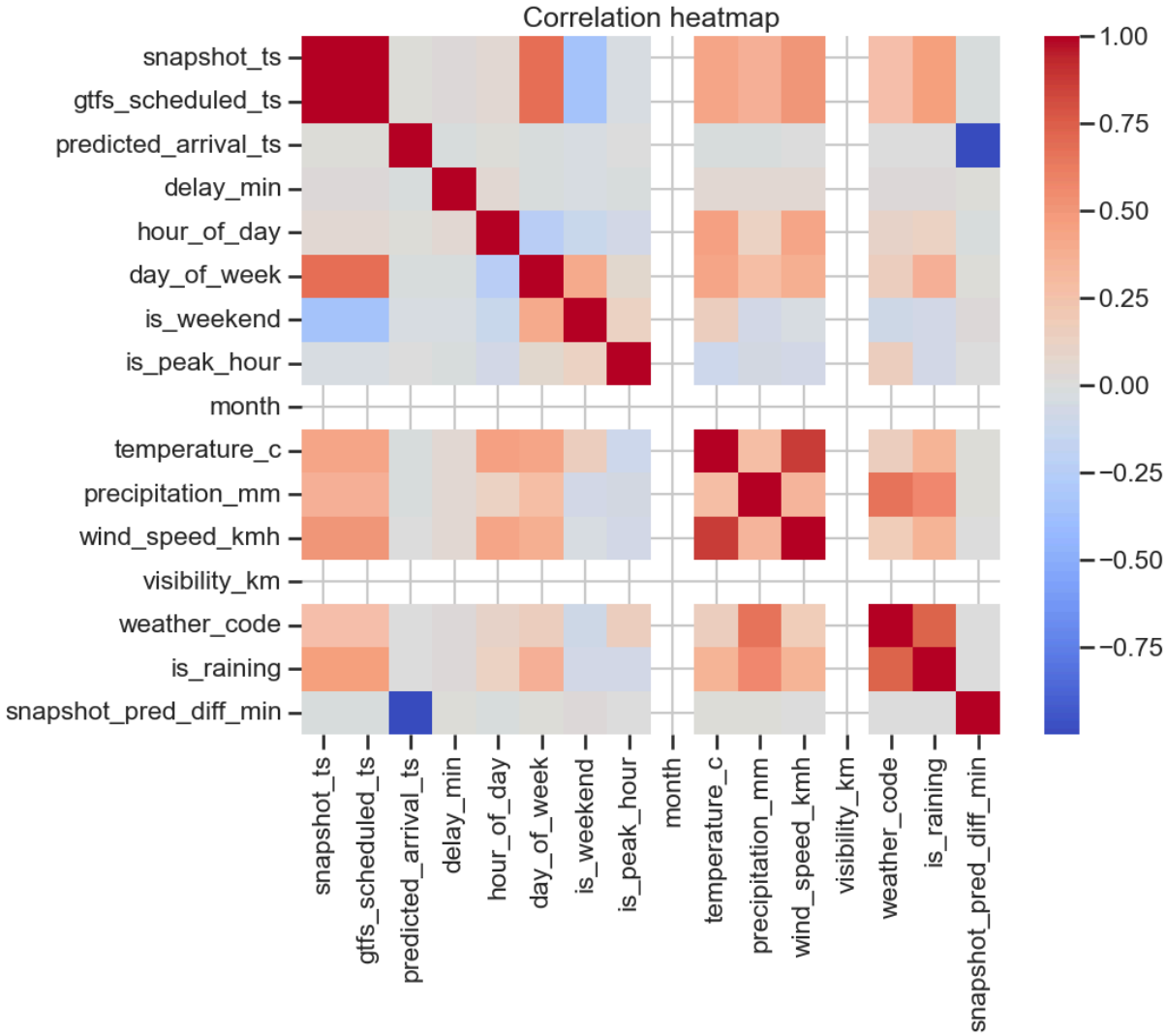
```
                   count       mean  median
route_short_name
F Line               658 -12.097705  -14.35
14                   350  -0.761200   -0.86
181                  345  -7.326406   -6.33
H Line               319  -0.694169   -1.03
550                  317  -4.603565   -4.98
2                    282 -10.017872   -8.43
7                    261  -0.513333   -0.82
40                   257   0.776809    0.20
60                   250   3.573600    2.34
150                  239  -0.941674   -1.43
D Line               233  -0.452618   -0.82
161                  229 -10.174891   -5.20
554                  225  -3.574889   -3.60
E Line               212  -1.759057   -1.49
372                  197  -0.654873   -0.83
```




In [7]:
```
# Time-of-day pattern
plt.figure(figsize=(10,4))
sns.lineplot(data=df, x='hour_of_day', y='delay_min', hue='is_raining', pale
plt.title('Delay by hour_of_day (rain vs dry)')
plt.xlabel('hour_of_day')
plt.ylabel('delay_min')
plt.legend(title='is_raining', labels=['dry','rain'])
plt.show()
```




In [8]:
```
# Correlation heatmap for numeric features
num_cols = df.select_dtypes(include=['float64','int64']).columns
corr = df[num_cols].corr()
plt.figure(figsize=(10,8))
sns.heatmap(corr, cmap='coolwarm', center=0, square=True)
plt.title('Correlation heatmap')
plt.show()
```




## Notes / To-do

- If snapshot vs predicted gap shows many values > ±5 min, revisit collector filtering and consider discarding those rows in post_process.
- Consider adding engineered features: route encoding, rain×peak interaction, rolling averages by route/hour.
- Re-run this notebook after any pipeline change to confirm distributions and sanity checks remain stable.